

Project Phase 2: Pipelining RISC-V Processor Design

Sri Pranavadarsh Tadepalli
2024102028
sripranavadarsh.t@students.iiit.ac.in

Prasanna NR
2024102056
prasanna.r@students.iiit.ac.in

Praveen V
2024102019
praveen.v@students.iiit.ac.in

Abstract—This project presents the design and implementation of a fully functional 64-bit RISC-V pipelined CPU. Building upon previous sequential processing project, this design implements a 5-stage in-order pipeline architecture. Core features include gate-level combinational logic for the ALU and control units, alongside comprehensive hazard handling via forwarding and stall logic. The processor successfully executes a subset of the RV64I instruction set, and functionality is verified through testbenches executing complex iterative programs, including Fibonacci sequence generation and linear search algorithms.

Index Terms—RISC-V, pipelining, RV64I, hazard handling, data forwarding, ALU, processor design

I. INTRODUCTION

Pipelining is a method to highly optimize the performance of a processor. It works by executing multiple instructions simultaneously in different stages of the processor. In a tradition sequential implementation the whole processor waits until an instruction is executed. In pipelining instructions are executed at multiple stages, allowing new instructions to enter the pipeline well before the previous ones finish.

There are considerable number of advantages to the pipelining implementation.

- 1) By overlapping execution, multiple instructions are completed in the same time period.
- 2) Each stage of processor is involved in a task at any given time rather than lying idle.
- 3) The overall latency is the same but the total number of cycles is reduced considerably.

II. PROJECT OBJECTIVES

The primary objective of this phase is to design and implement a functional pipelined RISC-V processor datapath. The specific goals include:

- Transitioning from a single-cycle sequential datapath to a 5-stage pipelined architecture.
- Implementing complete hazard handling, including load-use stalls and data forwarding (EX-EX, MEM-EX, and WB-ID same-cycle forwarding).
- Designing gate-level implementations for the ALU and Control Unit.
- Verifying correct pipeline execution, flush mechanisms, and cycle counts for arithmetic, memory, and branch instructions.

III. INSTRUCTION SUBSET AND ENDIANNESS

The processor supports a subset of the RISC-V RV64I instruction set architecture, maintaining a Big-Endian memory organization for both instruction and data memory to ensure uniform interpretation. The supported instructions include:

- R-Type: add, sub, and, or
- I-Type: addi, ld
- S-Type: sd
- B-Type: beq

IV. PIPELINE ARCHITECTURE

The CPU implements a 5-stage in-order pipeline separated by four pipeline registers: IF/ID, ID/EX, EX/MEM, and MEM/WB.

A. Instruction Fetch

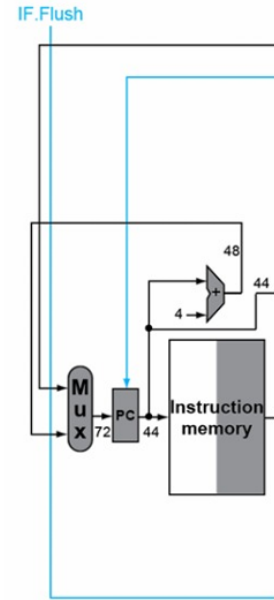


Fig. 1. IF stage

The 64-bit Program Counter (PC) supplies the address of the next instruction. The Instruction Memory (byte-addressable)

C. Execute Stage (EX)

B. Instruction Decode (ID)

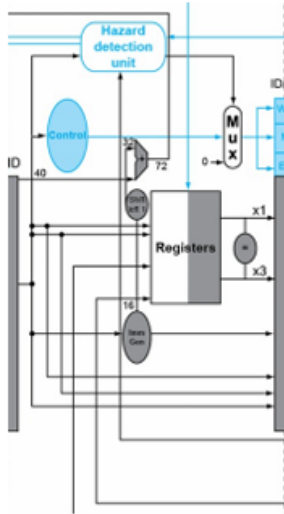


Fig. 2. ID stage

The four functions are Register read, Immediate generation, Control signal generation and Hazard detection.

The hazard unit is required to prevent the load-use dependencies. A load use hazard occurs when an instruction need data which hasn't been written back into the register i.e it is still in the pipeline.

Fig. 3. EX stage

The ALU receives two inputs, which are selected through multiplexers. The first operand is chosen between register value(rs1) and a forwarded value. The second operand is chosen between register value(rs2) and an immediate value.

The forwarding unit determines whether to use the values from previous pipeline stages to avoid stalls. This is used to handle the Data hazard conditions. To resolve the data hazards the forwarding unit makes sure that most recent data is used in execution without waiting for executing write back operation

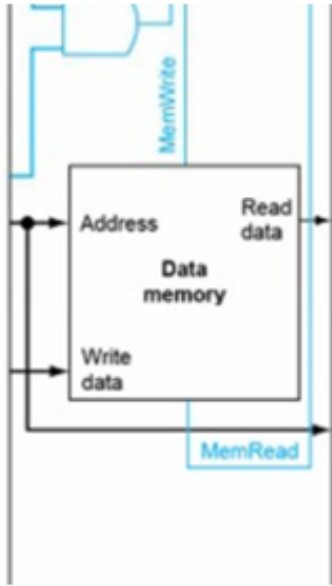


Fig. 4. MEM stage

The Data Memory (byte-addressable) handles 64-bit double-word reads and writes. It features synchronous writes on the positive clock edge and combinational reads. It is responsible for accessing data from memory for load and store instructions. This stage executes load/store functions.

The key components are Data memory, Branch logic and pipeline register.

Data memory stores and retrieves values for load/store instructions.

Branch logic uses the zero flag and branch condition to decide the PC update.

Pipeline register stores the imm values before passing onto the next stage.

The memory operations are controlled by MemRead and MemWrite. They enable reading/writing from/to the memory.

E. Write Back (WB)

The Writeback Mux selects between the ALU result and memory read data based on the MemtoReg signal, writing the final value back to the destination register.

The output from the memory stage is select based on the control signal of the write_back multiplexer. When the MemtoReg control is set to 1 load operation is executed or else the Alu result is written back.

The value which is selected is stored in two locations which are the destination register and the ID/EX register. The destination register holds the final result of the instruction execution while the ID pipeline register is used for forwarding and hazard resolution.

• IF/ID registers:

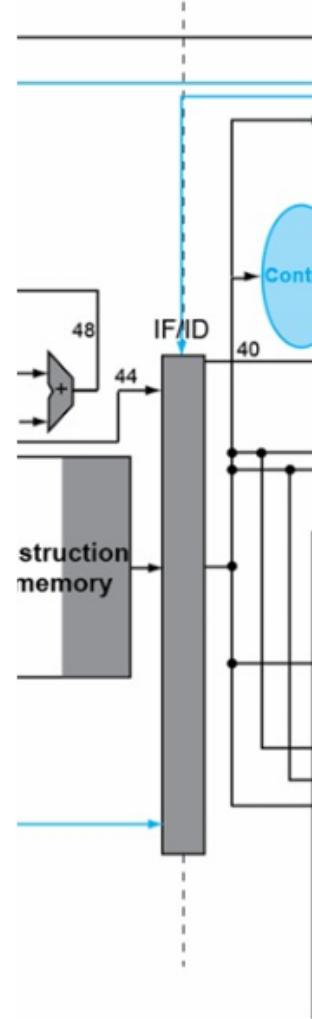


Fig. 5. IF ID register

The **IF/ID pipeline register** acts as the intermediate storage between the Instruction Fetch (IF) stage and the Instruction Decode (ID) stage in a pipelined processor. During the instruction fetch stage, the instruction is read from memory using the address stored in the Program Counter (PC) and is then placed into the IF/ID pipeline register. At the same time, the PC is incremented by 4 and written back into the PC so that the processor is ready for the next clock cycle. The incremented PC value is also stored in the IF/ID pipeline register in case it is needed later by instructions such as branch instructions (e.g., beq). Since the processor cannot determine the exact type of instruction during the fetch stage, it forwards all potentially required information to the next stage of the pipeline.

In our implementation, the IF/ID register also prepares the fetched instruction for decoding by extracting key

fields from the 32-bit instruction. The opcode field (bits [6:0]) is isolated to identify the operation type, which determines the category of the instruction. The destination register field `rd` (bits [11:7]) is extracted to specify the register that will receive the result. Similarly, the source register fields `rs1` and `rs2` are extracted to determine the registers that contain the input operands required for execution. These extracted values, along with the PC value and the full instruction, are passed to the next stage through the ID/EX pipeline register.

To manage pipeline hazards, the IF/ID register is controlled by the `IF_ID_Write` signal. The register accepts new instruction and PC values only when this signal is asserted; otherwise, it holds its current contents, effectively stalling the pipeline. Additionally, when a control hazard occurs, such as during a taken branch, the pipeline is cleared using an `IF_Flush` mechanism driven by the `PCsrc` signal to remove incorrect instructions from the pipeline.

- **ID/EX pipeline register:**

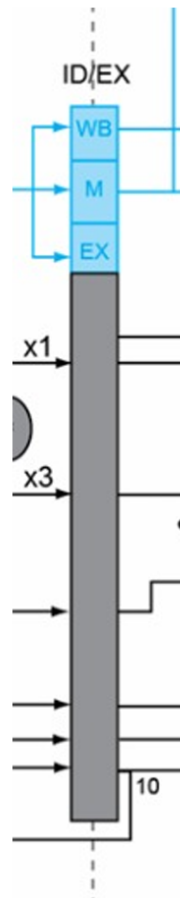


Fig. 6. ID EX register

signals needed for the next stage.

- The register stores the following information:
 - * Register Operands: The values read from the register file corresponding to the source registers (`rs1_data` and `rs2_data`).
 - * Immediate Value: The sign-extended immediate generated from the instruction for ALU operations.
 - * Program Counter (PC): The address of the current instruction used for branch target calculations.
 - * Control Signals: Control flags generated by the control unit such as `ALUOp`, `MemRead`, `MemWrite`, `RegWrite`, and `Branch`.
 - * Register Identifiers: The indices of the source and destination registers (`rs1`, `rs2`, and `rd`) required for forwarding and hazard detection.
- In addition, the ID/EX register transfers decoded instruction information from the Instruction Decode stage to the Execute stage and stores:
 - * Program counter value
 - * Instruction
 - * Register operand values
 - * Immediate value
 - * Source and destination register indices
 - * Control signals such as `MemRead`, `MemWrite`, `RegWrite`, `ALUSrc`, and `Branch`
 - * Branch prediction information (`pred_taken` and prediction index)
- The register also handles hazard-related control signals:
 - * When `reset` or `flush` is asserted, all outputs are cleared to insert a bubble into the pipeline.
 - * When `stall` is asserted, the control signals are cleared while the data signals are preserved, effectively inserting a pipeline bubble while maintaining the correct data flow.
- Branch prediction information is forwarded to the Execute stage through this register so that the processor can later determine whether the prediction was correct.

- Acts as the pipeline register between the Instruction Decode (ID) and Execute (EX) stages, storing all required operands, immediate values, and control

- **EX/MEM Register:**

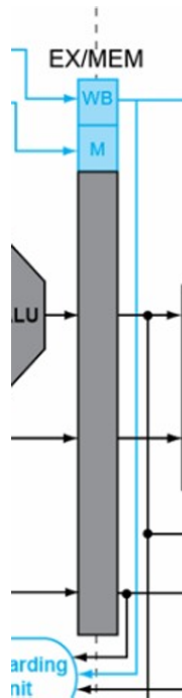


Fig. 7. EX MEM register

- Acting as the bridge between the Execution (EX) and Memory Access (MEM) stages, the EX/MEM register captures the results produced by the ALU along with the control signals required for the next stage of the pipeline. It stores computation results, branch evaluation information, and data required for memory operations.
- This register also assists in hazard resolution. By providing the destination register identifier (EX_MEM_rd), the forwarding unit can detect data dependencies and forward results to subsequent instructions when necessary.
- The register holds the following data:
 - * ALU Result: The output of the arithmetic or logical operation performed in the EX stage.
 - * Branch Target Address: The computed address for branch instructions.
 - * Zero Flag: Indicates whether the ALU result is zero and is used for branch decision logic.
 - * Store Data: The value forwarded from the ID/EX stage that will be written to memory during store instructions.
 - * Destination Register (rd): The register index that will receive the final result during the write-back stage.
 - * Control Signals: Signals required for later stages, including memory access and write-back control such as MemRead, MemWrite, MemtoReg, and RegWrite.

- The MEM/WB register transfers the final results from the Memory (MEM) stage to the Write Back (WB) stage.
- The following information is stored in this register:
 - * Data read from memory
 - * ALU result
 - * Destination register index
 - * Write-back control signals such as MemtoReg and RegWrite
- During the Write Back stage, the value written to the register file is selected based on the MemtoReg signal. If MemtoReg is asserted, the data read from memory is written back; otherwise, the ALU result is written to the destination register.

- **MEM/WB Register:**



Fig. 8. MEM WB stage

- The MEM/WB register temporarily holds the ultimate results and control directives right before the Write-Back (WB) phase. Its primary job is to guarantee that the final data is accurately saved back into the processor's register file.
- It receives the following inputs from the memory stage to complete the instruction cycle:
 - * ALU Output: The finalized result from the execution stage's math or logic operations.
 - * Fetched Memory Data: Information successfully retrieved from main memory.

- * Write Authorization (RegWrite): The control flag that grants permission to actually update the register file.
- * Data Source Selector (MemtoReg): The control switch that decides which piece of data—the ALU’s calculation or the fetched memory value—gets written to the destination.
- * Target Address (rd): The identifier for the specific destination register that will receive the final saved value.

VI. HAZARD HANDLING AND FORWARDING

To ensure correct execution without stalling unnecessarily, the processor implements several hazard mitigation strategies:

- Load-Use Data Hazard (Stall): If a ld instruction is followed by a dependent instruction, the PC and IF/ID registers freeze, and a NOP bubble is inserted into the ID/EX register.
- EX-EX Forwarding: If an R-type or I-type instruction writes a result needed by the immediate next instruction, the ALU result from EX/MEM is forwarded directly to the ALU input.
- MEM-EX Forwarding: If a result from the MEM/WB stage is needed by the current EX stage, the writeback mux output is forwarded to the ALU input.
- Branch Flush (Control Hazard): Branch outcomes are resolved in the EX stage. If a branch is taken, both IF/ID and ID/EX registers are flushed to zero, incurring a 2-cycle penalty.

VII. SIMULATION AND VERIFICATION

Test case to demonstrate load-use stall:

```
ld x1, 0(x5)
add x2, x1, x3
```

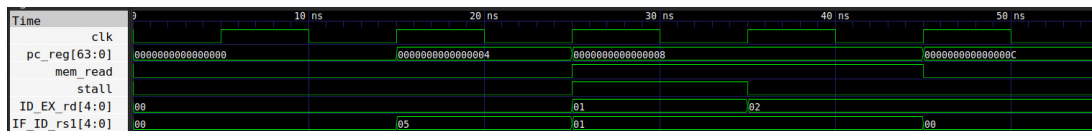


Fig. 9. Waveform demonstrating load-use stall in the pipeline

The waveform shows a **load-use data hazard**. When the ld instruction is in the pipeline, mem_read = 1 and ID_EX_rd = 01 (destination register x1). At the same time, the following add instruction has IF_ID_rs1 = 01, meaning it requires the value of x1. Since the data from the load is not yet available, the hazard detection unit asserts stall = 1, inserting a bubble in the pipeline until the value is ready.

Test case to verify pipeline flushing during branch:

```
addi x1, x0, 5
addi x2, x0, 5
beq  x1, x2, 8
addi x3, x0, 100
addi x4, x0, 200
addi x5, x0, 10
add  x6, x5, x1
beq  x0, x0, 0
```

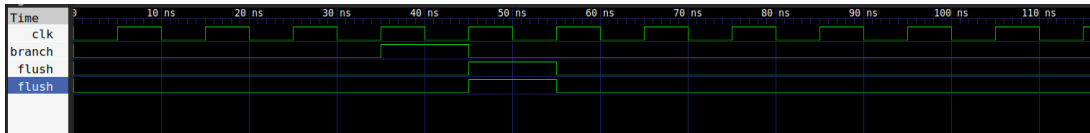


Fig. 10. Waveform showing pipeline flush after branch resolution

The waveform shows **branch flushing**. When the branch signal becomes high, the control unit asserts the `flush` signal, clearing the incorrectly fetched instructions from the pipeline.

Test case to verify data forwarding:

```
add x1, x2, x3
sub x4, x1, x5
```

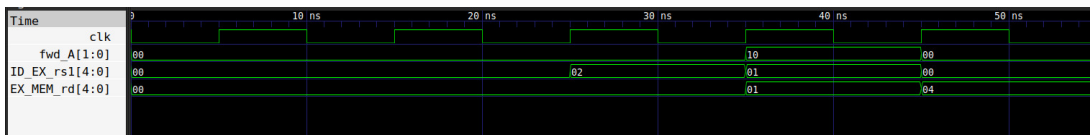


Fig. 11. Waveform showing EX-to-EX data forwarding

The waveform illustrates **data forwarding**. The register required by the current instruction in the EX stage (`ID_EX_rs1`) matches the destination register of the previous instruction in the MEM stage (`EX_MEM_rd`). As a result, the forwarding unit sets `fwd_A = 10`, indicating that the operand is forwarded directly from the EX/MEM stage instead of waiting for write-back. This resolves the data dependency without stalling the pipeline.

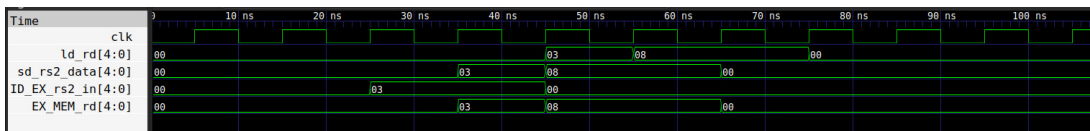


Fig. 12. Waveform showing load-to-store forwarding

The waveform shows **load-to-store forwarding**. The destination register of the load instruction (`EX_MEM_rd`) matches the source register of the store instruction (`ID_EX_rs2`). The loaded value is forwarded directly to the store data path (`sd_rs2_data`) without waiting for the write-back stage, preventing a pipeline stall.

VIII. BRANCH PREDICTION LOGIC

To reduce the performance penalties associated with control hazards, a tournament branch predictor was implemented. This architecture dynamically selects the most reliable prediction between a 1-bit predictor and a 2-bit saturating counter, guided by a tournament selector.

A. Architecture Overview

The prediction logic is integrated into the Fetch (IF) stage, providing a predicted path for the program counter (PC) before the branch instruction resolved. The prediction code consists three main components:

- **1-bit Predictor:** A simple state machine that gives the result of the last branch execution, if the prediction was wrong then it replaces the bit in the table by what ever actually happened very effective for simple loops but prone to oscillation in complex instruction patterns.
- **2-bit Saturating Counter:** A more reliable predictor in most of the scenarios that requires two consecutive mispredictions before switching states.
- **Tournament Selector:** A 2-bit counter for each entry that tracks the performance history of the two sub-predictors and selects the more reliable source.

TABLE I
TOURNAMENT SELECTOR UPDATE LOGIC

Scenario	1-bit Result	2-bit Result	Action
Both are Right	Correct	Correct	No Change.
Both are Wrong	Wrong	Wrong	No Change.
1-bit is Better	Correct	Wrong	Decrease Score.
2-bit is Better	Wrong	Correct	Increase Score.

B. Training and Feedback Loop

The prediction state is treated as transient meta-data (information about information). When a branch is predicted, the state of the 1-bit and 2-bit predictors is passed through the pipeline stages (IF/ID and ID/EX) to maintain synchronization with the specific instruction instance.

Upon reaching the Execute (EX) stage, the actual outcome of the branch is calculated. The predictor update logic performs the following:

- 1) **Correction:** If a misprediction occurs, the pipeline is flushed, and the PC is corrected.
- 2) **State Update:** The sub-predictors update their internal counters based on the actual result.
- 3) **Selector Adjustment:** The tournament selector is updated only when the two sub-predictors provide conflicting results, reinforcing the predictor that correctly guessed the outcome.

C. Implementation

The implementation utilizes a 64-entry table indexed by $PC[7 : 2]$. The logic is defined by the following state transitions for the 2-bit saturating counter:

$$State_{next} = \begin{cases} 00, & \text{if } State = 01 \text{ and } Taken = False \\ 00, & \text{if } State = 00 \text{ and } Taken = False \\ 01, & \text{if } State = 00 \text{ and } Taken = True \\ 10, & \text{if } State = 01 \text{ and } Taken = True \\ 01, & \text{if } State = 10 \text{ and } Taken = False \\ 11, & \text{if } State = 10 \text{ and } Taken = True \\ 10, & \text{if } State = 11 \text{ and } Taken = False \\ 11, & \text{if } State = 11 \text{ and } Taken = True \end{cases} \quad (1)$$

Basically the 2-bit counter moves toward 11 if the branch is taken and it moves toward 00 if the branch is not taken similar to the tournament selector which moves toward 2-bit predictor if the results provided by both the sub-predictors are conflicting and 2-bit predictor is right and moves towards 1-bit predictor if the results are conflicting and 1-bit is right.

The MSB bit for 2-bit sub-predictor serves as output of that sub-predictor. The only bit in a 1-bit predictor serves as the output for that subpredictor.

The 1-bit sub-predictor flips its state only when its prediction is wrong.

In the tournament selector counter we take the prediction of 1-bit sub-predictor if the counter is 0 or 1 and we take the prediction of the 2-bit sub-predictor if the counter is 2 or 3.

If any of the counters get saturated it remains saturated i.e it is neither increased nor decreased.

We assume that state 0 as not taken and 1 as taken.

The mapping of the Branch memory is direct i.e if 65th branch instruction comes it gets updated to 0th indexed memory. In this way we can keep track of 64 branch instruction at single time which is more than enough at this level.

To ensure temporal consistency, prediction metadata is propagated through the pipeline via the IF/ID and ID/EX registers. This 'pass-through' mechanism ensures that when a branch reaches the Execute (EX) stage for verification, the original states of the 1-bit, 2-bit, and tournament selector counters are available. This allows the update logic to compare the actual branch outcome against the specific speculative state that is generated as prediction and modify the prediction table.

As usual if the prediction turns out to be wrong we flush the 2 instructions which are fetched after the branch instruction.

IX. CONCLUSION

This project successfully achieved the design and implementation of a fully functional 32-bit RISC-V pipelined CPU. By transitioning from a sequential architecture to a 5-stage in-order pipeline, we demonstrated the effectiveness of overlapping instruction execution to enhance overall processor performance. In summary, this work highlights the critical balance between pipeline depth, hazard mitigation, and speculative execution, providing a solid foundation for further development in high-performance computer architecture.

X. WORK DISTRIBUTION

Person	Name	Work
1	Prasanna	Instruction Memory, Control Unit, Pipeline datapath and pipeline registers, Integration of all modules
2	Pranav	Register File, Immediate Generator (ImmGen), Forwarding Unit, Debugging, Pipeline report
3	Praveen	Data Memory, ALU Control, Hazard Detection Unit and Branch Flush, Debugging, Sequential (SEQ) report

XI. ACKNOWLEDGMENT

We would like to sincerely thank Prof. Priyesh Shukla and TA's for their Guidance throughout the Course.